

CHAPTER 3

Verilog 的模組 與架構



在本章將為各位介紹：Verilog 的模組架構，Verilog 的輸出入埠敘述，包括輸入埠(input)、輸出埠(output)、雙向埠(inout)等三種敘述，另外也特別說明 Verilog 對於輸出入埠的連接規定，作一個簡要的說明。

Verilog 的資料型態(Data Types)以及時間控制(Timing Control)方式。

Verilog 的四大模型(Model)、Verilog 的模組(Module)、Verilog 的語法協定，以及階層式設計(Hierarchy Design)的觀念。



3.1 Verilog 的輸出入埠敘述

Verilog 的輸出入埠敘述，包括輸入埠(input)、輸出埠(output)、雙向埠(inout)等三種敘述，另外也特別說明 Verilog 對於輸出入埠的連接規定，作一個簡要的說明。

3.1.1 Verilog 的模組架構

Verilog 的模組架構，包括：以 module 這個字眼開始，接著是 module_name 模組名稱(例如：adder、mux、counter...等等)，再來(輸出入埠名稱)，模組的內容則有輸出入埠敘述、資料型態敘述以及電路功能的描述，而模組結束前必定要有 endmodule 這個字眼。

```
module  module_name (輸出入埠 名稱)
    輸出入埠 敘述
    資料型態 敘述
    電路功能 敘述
endmodule
```

▲ Verilog 的模組架構

3.1.2 input（輸入埠）敘述

input（輸入埠）：所定義的埠，具有輸入埠的特性。

BNF 表示法

```
<input_declaration>  
:= input <range>? <list_of_variables>;
```

3.1.3 output（輸出埠）敘述

output（輸出埠）：所定義的埠，具有輸出埠的特性。

BNF 表示法

```
<output_declaration>  
:= output <range>? <list_of_variables>;
```

例：input（輸入埠）及 output（輸出埠）敘述的 Verilog 片段範例。

```
input    [3:0] A, B; // 宣告二個具有 4 個位元的輸入埠 A 及 B  
output   [3:0] Z; // 宣告一個具有 4 個位元的輸出埠 Z  
reg      [3:0] Z; // 宣告一個具有 4 個位元的暫存器型態變數 Z  
  
always @(A or B)  
begin  
    Z = A + B;  
end
```

3.1.4 inout（雙向埠）敘述

所定義的埠，既有輸入埠的特性也有輸出埠的功能。

BNF 表示法

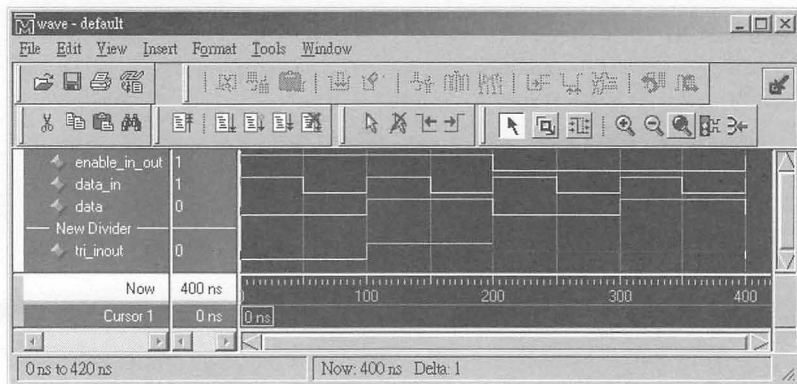
```
<inout_declaration>
:= inout<range>? <list_of_variables>;
```



▲ BiDir.V：雙向埠 (inout, Bidirectional) 驅動的邏輯符號

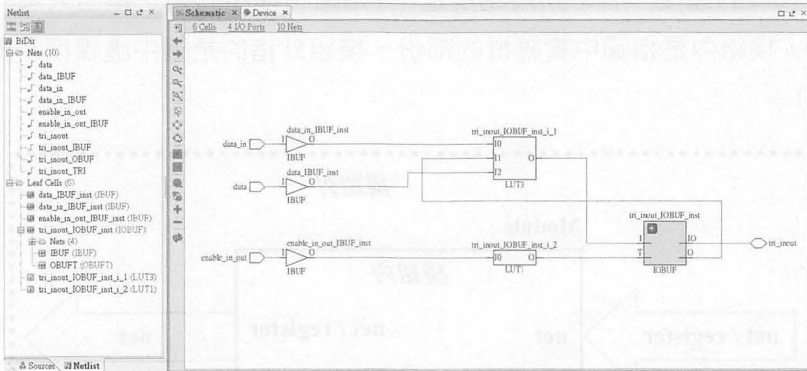
```
// BiDir.V, Bidirectional 驅動範例
// 如果 enable_in_out = 1 且 data_in = 1, 則 tri_inout = data
// 如果 enable_in_out = 1 且 data_in = 0, 則 tri_inout 的值不變
// 如果 enable_in_out = 0, 則 tri_inout = 1'bz
module BiDir (enable_in_out, data_in, data, tri_inout);
input    enable_in_out, data_in, data;
inout    tri_inout;
wire     tri_inout;

    assign tri_inout= enable_in_out ? ((data_in) ? data :
tri_inout) : 1'bz;
endmodule
```

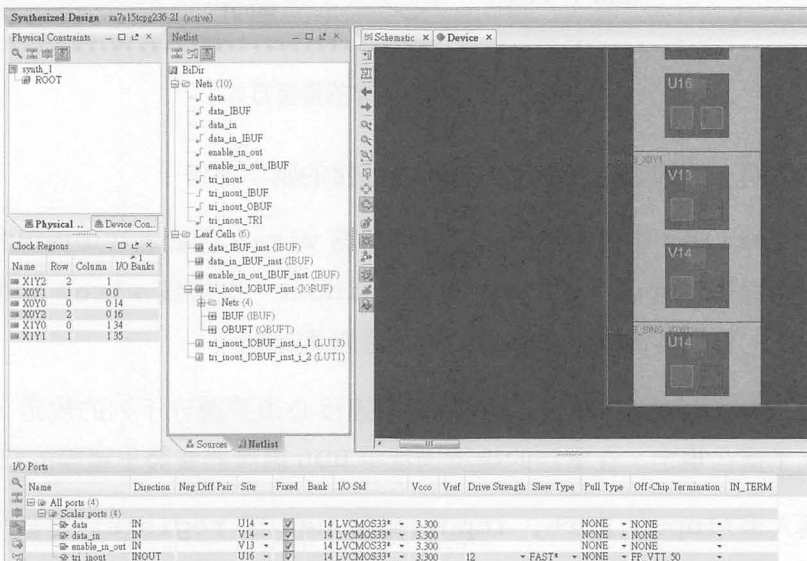


▲ BiDir.V 的模擬結果

下圖是將 BiDir.V 電路描述經由 Xilinx 的 Vivado 合成出來的電路 (Schematic), Nets 和 Left Cells 連接情形。

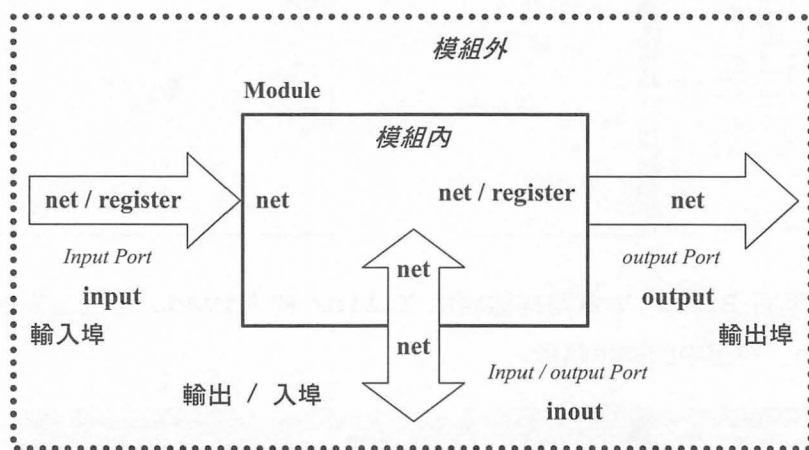


下圖是將 BiDir.V 電路描述經由 Xilinx 的 Vivado 合成出來的電路在「Device」視窗中的顯示情形。



3.1.5 輸出入埠的連接規定

輸出入埠的連接，有下列幾點規定：（如圖「模組內外之輸出入埠的連接方式」所示，模組內是指圖中實線框的部份、模組外指的是圖中虛線框及實線框之間的部份）。



▲模組內外之輸出入埠的連接方式

- 可以將一個埠視為在模組內外相互連接的兩個部份。
- 在 Verilog 中內定的埠之宣告種類為 `wire`，因此若在埠與埠的宣告中只有宣告 `input`、`output` 或者是 `inout`，則視為 `wire`；如果需要將訊號的值儲存起來就要將埠的種類宣告為 `reg`。
- 在 Verilog 中埠的內部與外部的連接必須要遵守下列的規定，若是違反了這些規定，在電路的編譯過程中 HDL 編譯器會發出錯誤的訊息。
- 輸入埠 (Input Port, `input`) 能被 `net` 或 `register` 推動，而一個輸入埠 (Input Port, `input`) 能夠推動 `net` 型態的接線。
- 輸出埠 (Output Port, `output`) 能被 `net` 或 `register` 推動，而一

個輸出埠 (Output Port, output) 能夠推動 net 型態的接線。

- 雙向埠 (Inout Port, inout) 能被 net 推動，而一個輸出埠 (Inout Port, inout) 能夠推動 net 型態的接線。
- 可用於電路合成的“net”（接線型態），包含下列三種：
 - ① wire 敘述
 - ② wor 敘述 (Wired-OR 型態接線)
 - ③ wand 敘述 (Wired-AND 型態接線)



3.2 Verilog 的資料型態 (Data Types)

Verilog 的資料型態 (Data Types)，主要包括：wire (接線) 敘述、wand (Wired-AND 型態的接線)、wor (Wired-OR 型態的接線) 敘述，以及 reg (暫存器) 敘述所定義出來的輸出入埠值和變數的值，而四種數值位準 (Four Value Level) 則是由外界傳入到輸入埠值、模組將結果傳出到輸出埠值，以及模組內部變數值的邏輯準位。

3.2.1 四種數值位準 (Four Value Level)

Verilog 所提供的四種數值位準，如下表所示。

定義這四種數值位準的目的是為了解決在實際電路中，二個不同數位準的驅動訊號 (driver) 發生了衝突 (conflict)，那麼被驅動之訊號最後的值為何？則取決於這二個不同數位準的驅動訊號所要驅動的目的地訊號之型態。目的地訊號之型態，可以是 wire、wand 或 wor 等接線型態。這三種接線型態的目的在於提供解決多條輸入線連接到同一條輸出線時所會產生的邏輯矛盾 (Logical Conflict Resolution) 的現象。

數值位準	實際電路狀態
0	邏輯 0、Zero、假 (False)、Low、Logic Low、Ground、Vss、Negative Assertion
1	邏輯 1、One、真 (True)、High、Logic High、Power、VDD、Vcc、Positive Assertion
x	不確定值 (Unknown Value)
z	高阻抗 (High Impedance)、浮接狀態 (Floating State)、Tri-Stated、Disabled Driver (Unknown)

3.2.2 wire (接線) 敘述

wire (接線) 的特性，如下：

- 接線是連接硬體元件之連接線。
- 接線必須要被驅動才能改變它的內涵值。
- 接線描述的關鍵字為 wire。
- 除非有被宣告成一個向量 (vector)，否則接線是內定為一個位元的值，而且內定值是 z。

BNF 表示法

```
<net_declaration>
:= <NETTYPE> <charge_strength>? <expandrange>? <list_of_variables>;
|| <NETTYPE> <charge_strength>? <expandrange>? <list_of_variables>;
<NETTYPE> := wire
<expandrange> := <range>
```

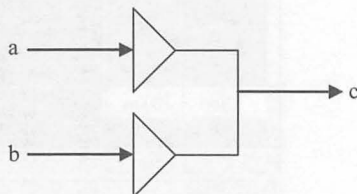
例：

```
wire b, c; // 宣告 2 條接線，且命名為 b 及 c
wire a = b & c; // 宣告 1 條接線 a，並指定它為 b & c
wire d = 1'b0; // 宣告 1 條接線 d，並指定它的值為一個固定值 0
```


3.2.3 wand (Wired-AND 型接線)、wor (Wired-OR 型接線) 敘述

(一) wand (Wired-AND 型態接線)、wor (Wired-OR 型態接線) 的特性

- 我們不能將“多條”訊號輸出線同樣連接到使用 wire 型態的某一條接線。
- 若將“多條”訊號輸出線同樣連接到使用 wire 型的某一條接線，則 Quick Logic 公司所設計出來的 SpDE 在編譯時會出現下列的錯誤訊息：Mapper Error: Aborting because of driver errors.
- 可以將“多條”訊號輸出線同樣連接到某一條 wand 或者是 wor 這些型態的接線，這兩種接線的目的在於提供解決將“多條”訊號輸出線同樣連接到使用 wire 型態的某一條接線時所會產生的邏輯矛盾(Logical Conflict Resolution)的現象。



(二) wire 真值表

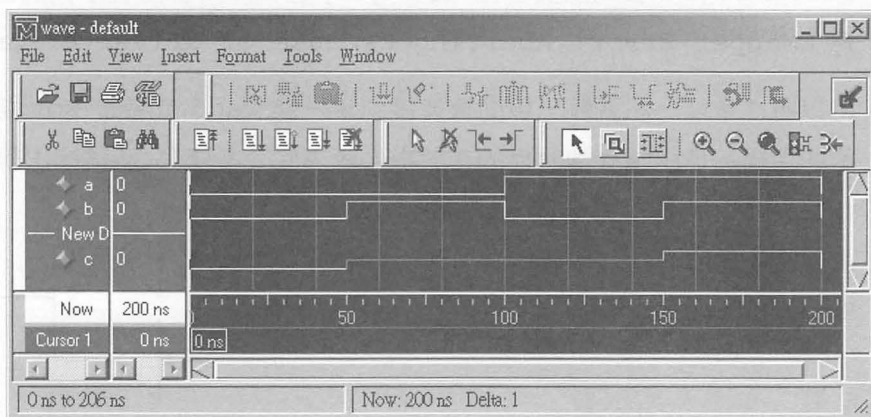
wire					
c		a			
		0	1	X	Z
b	0	0	X	X	0
	1	X	1	X	1
	X	X	X	X	X
	Z	0	1	X	Z

例：WIRE.V 範例經由 Quick Logic 公司所設計出來的 SpDE 編譯過程，

會出現下列的錯誤訊息：Mapper Error: Aborting because of driver errors.

```
// WIRE.V: Wire 敘述，錯誤的範例
module wire_test(a, b, c);
input  a, b;
output c;
wire  c;

assign c = a;
assign a = b;
endmodule
```



▲ WIRE.V 的模擬結果

(三) wand 真值表

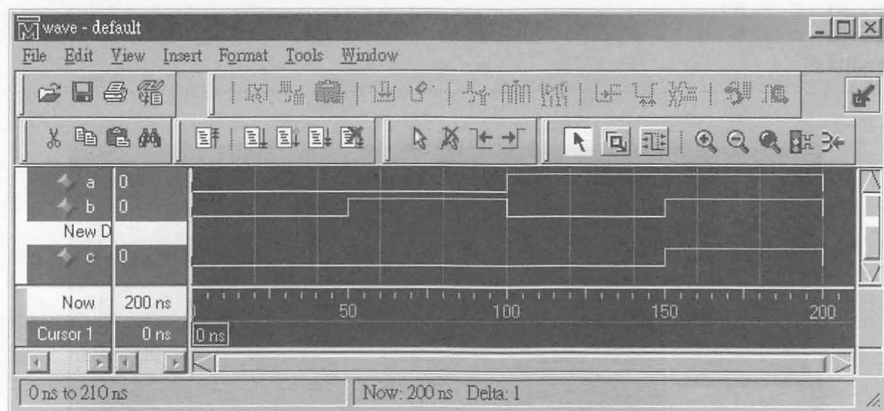
在 VLSI 的技術上，是用「開集極」(open collector)元件來達到 wired-AND 的功能。

wand					
c		a			
		0	1	X	Z
b	0	0	0	0	0
	1	0	1	X	1
	X	0	X	X	X
	Z	0	1	X	Z

例：WAND.V, Wired-AND 敘述，範例

```
// WAND.V:Wired-AND 敘述，範例
module wand_test(a, b, c);
input  a, b;
output c;
wand   c;

    assign c = a;
    assign c = b;
endmodule
```



▲ WAND.V 的模擬結果

(四) wor 真值表

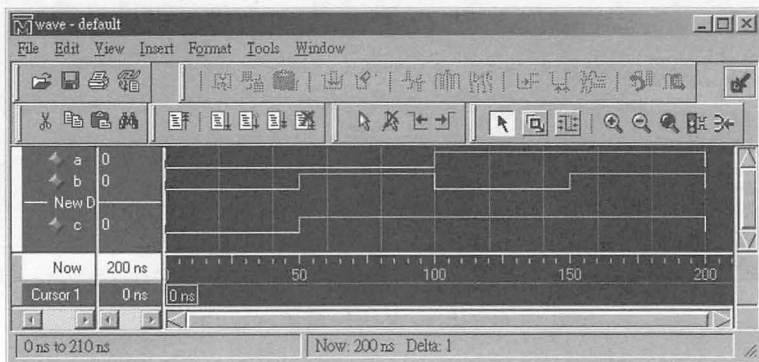
在 VLSI 的技術上，是用「射極耦合邏輯」(ECL, Emitter Couple Logic) 元件來達到 wired-OR 的功能。

wor					
C		A			
		0	1	X	Z
B	0	0	1	X	0
	1	1	1	1	1
	X	X	1	X	X
	Z	0	1	X	Z

例：WOR.V, Wired-OR 敘述，範例

```
// WOR.V: Wired-OR 敘述，範例
module wor_test(a, b, c);
input  a, b;
output c;
wor    c;

    assign c = a;
    assign c = b;
endmodule
```



▲ WOR.V 的模擬結果

3.2.4 reg (暫存器) 敘述

reg (暫存器) 的特性如下：

- 暫存器 (reg) 的功能與變數很像，可以直接給定一個數值，主要的功能在於保持住電路中的某個值；不必像接線 (wire) 要被驅動才能改變它的內函值。
- Verilog 的暫存器 (reg) 描述與硬體的暫存器 (Registers) 不盡相同。
- 暫存器描述的關鍵字為 reg。
- 除非有被宣告成一個向量 (vector)，否則接線是內定為一個位元的值，而且內定值是 x (don't care)。

BNF 表示法

```
<reg_declaration>  
:= reg <range>? <list_of_regster_variables>;
```

例：

```
input      [3:0] sum;  
output     zero; // 宣告 1 個輸出埠，且命名為 zero  
reg        zero; // 宣告 1 個暫存器，且命名為 zero  
  
always @(sum)  
begin  
    if(sum == 0) // 如果 sum = 0  
        zero = 1; // 則，令 zero 一直保持為 1  
    else  
        zero = 0; // 否則，令 zero 一直保持為 0  
end
```

3.2.5 選用 wire (接線) 或 reg (暫存器) 敘述的時機

選用 wire (接線) 或 reg (暫存器) 敘述的時機，說明如下：

- “wire” 必須配合 “assign” 來使用，且不能出現在 “always” 的區塊描述裡。

例：

```
wire a, b, c;

assign a = b & c;
```

- “reg” 的使用方法為 “a = b” 格式，也就是必須放在 “always” 的區塊描述裡。

例：

```
input [3:0] a, b;
output [3:0] z;
reg [3:0] z;

always @(a or b)
begin
    z = a + b;
end
```

3.2.6 向量 (Vectors) 表示法

向量 (Vectors) 表示法，說明如下：

- “wire” 和 “reg” 皆可定義為向量，若無定義位元長度，則視為被宣告的變數其位元長度為 1，也就是一個位元。
- 向量是一個多位元的元件 (element)。
- 向量可定義為 [High#:Low#] 或 [Low#:High#]。以 [High#:Low#] 來

說，在中括號內之分號左邊的部份為：最高位元 (Most Significant Bit, MSB)，在中括號內之分號右邊的部份為：最低位元 (Least Significant Bit, LSB)。

- 我們可以用向量的表示法，來取得向量內之部份訊號。

例：

```
wire even; // 宣告 1 條接線 even
wire bus [15:0]; // 宣告 1 條由 16 條接線所形成的匯流排，稱為 bus
reg bit; // 宣告 1 個 bit 的 reg
reg Result [31:0]; // 宣告 1 個用 32 個 reg 所形成的 register

assign even = ~ Result[0]; // 只取 Result 的第 0 個位元
assign bus[15:0] = Result[15:0]; // 只取 Result 的第 15 至第 0 個位元
```

3.2.7 陣列 (Arrays) 表示法

陣列 (Arrays) 表示法，說明如下：

- 陣列的內容可以是：整數、暫存資料，以及向量。
- HDL 編譯器只能用於描述單維陣列的表示法，不能描述多維陣列。
- 「陣列」是多個 1 位元或若干個位元的元件。

例：

```
reg [15:0] Word; // 1 個 Register, 這個 Register 為 16 個 Bits
reg Mem1Bit [1023:0]; // 1024 個 1 Bits 的 Register
reg [7:0] RegFile [1023:0]; // 1024 個 Register, 每個 Register 為 8 個 Bits
```

3.2.8 記憶體 (Memory) 表示法

記憶體 (Memory) 表示法，說明如下：

- 在 Verilog 中，記憶體像是一個暫存器的陣列，在陣列中的每一個物件就像是一個 Word，每個 Word 可以是一個位元或多個位元。
- N 個 1 Bit 的暫存器和一個 N Bits 的暫存器是不相同的。
- 陣列中的註標 (SubScript) 用於標名記憶體中的位址，用於指定我們想要存取某個記憶體元件 (暫存器)。

例：

```
reg Bit1, Bit2;
reg Memory1Bit [1023:0]; // 1024 個 1 Bit 的暫存器空間
reg [7:0] Byte1, Byte2; // 8 Bits
reg [7:0] Memory1Byte [1023:0]; // 1024 個 1 Byte (8 Bits)
    的暫存器空間

// 提取位址為 321 的 1 Bit 的暫存器給 Bit1
Bit1 = Memory1Bit [321];
// 將 Byte1 的值存入位址為 586 的 Memory1Byte 中
Memory1Byte [586] = Byte1;

// 提取在位址為 125 的 Memory1Byte 中之內容的第 4 個位元給 Bit2，
// 必須藉由二次提取的方式才可以完成
Byte2 = Memory1Byte [125]; // 首先：提取位址為 Memory1Byte [125]
Bit2 = Byte2 [4]; // 然後：提出第 4 個位元
```

3.2.9 位元選擇 (Bit-Selects) 與部份選擇 (Part-Selects)

位元選擇 (Bit-Selects) 與部份選擇 (Part-Selects) 是指選擇某個由向量 (Vectors) 表示法或是陣列 (Arrays) 表示法的變數之某些特定的部份。

(一) 位元選擇 (Bit-Selects)

位元選擇是選擇來自於 1 條 wire (接線)、1 個 register (暫存器) 或 parameter vector (參數向量) 的單一位元。

例：

```
wire [7:0] a, b, c;  
assign c[0] = a[0] & b[0]; // 選擇 a[0] 和 b[0] 作 & 後, 給 c[0]
```

(二) 部份選擇 (Part-Selects)

部份選擇是選擇來自於 wire (接線)、register (暫存器) 或 parameter vector (參數向量) 的一群位元。

例：

```
wire [7:0] a, b, c;  
// 選擇 a[7:0] 和 b[7:0] 作 & 後, 給 c[7:0]  
assign c[7:0] = a[7:0] & b[7:0];
```



3.3 Verilog 的時間控制 (Timing Control)

時間控制主要的用途在於設定某一個程序在特定的時間被執行。

可用於「可合成的 Verilog 電路描述」的時間控制方法，只有：事件基礎時間控制 (Event-based Timing Control)。

(一) 何謂「事件」？

- 當一條接線 (wire, wor, wand) 或暫存器 (register) 的值被改變時就是一個事件。
- 當模組的輸入埠接收到新的值時也是一個事件。
- 事件可以用來觸發一個敘述或一個內含多個敘述的區塊。

(二) 可用於「可合成的 Verilog 電路描述」的事件基礎時間控制

■ 正規事件控制 (Regular Event Control)

事件的控制符號是「@」，當信號產生正緣 (posedge)、負緣 (negedge)、轉換 (transition) 或者值被改變時，敘述才會被執行。

例：

```
always @(Clock) // 每當 Clock 的訊號有變化時
    q1 = d1;

always @(posedge Clock) // 當 Clock 的訊號由 0 變為 1 時
    q2 = d2;

always @(negedge Clock) // 當 Clock 的訊號由 1 變為 0 時
    q3 = d3;
```

■ 事件「或」控制 (Event OR Control)

當有多個訊號或事件觸發一個敘述或一個內含多個敘述的區塊，這種寫法如同將這些訊號作「或」的運算。

這種具有多個觸發訊號是用“or”這個關鍵字來指定的。

例：

```
// 非同步重設位準感測門
always @(Reset or Clock or d)
begin
    if (Reset) // 如果 Reset = 1, 則 q = 0
        q = 1'b0;
    else
        if (Clock) // 否則：如果 Clock = 1, 則 q = d
            q = d;
end
```



3.4 Verilog 的四大模型 (Model)

四大模型(Four kinds of Model)是指 Verilog 用以描述電路功能或是電路架構的四種表示方法。因此對於一個模組其內部的描述在 Verilog 中有四種不同的層次，設計的人可以依據不同的需要而使用不同的層次來描述模組的功能或內部電路。

一個模組外部所表現出來的功能不受內部描述所用層次的影響，也就是說對於一整個電路而言每個模組內部的詳細構造是隱藏無法得知的。因此對於模組內部所使用的描述層次我們可以任意的改變使用層次而不會影響到整個電路。

在實際的電路設計中，我們可以四個不同的層次混合使用。以下概述這四個層次：

(一) 低階交換模型 (Switch Level Model) 或電晶體層次 (Transistor Model)

這個層次是 Verilog HDL 中最低階的層次，電路是由開關與儲存點所組成。使用這個層次設計的工程師必須要知道電晶體的元件特性。

(二) 邏輯閘層次模型 (Gate Level Model)

在這個層次中，模組是由最基本的邏輯閘連接而形成的。在電路面積要求的情況下，可以先將所需的電路化簡，再使用現有的邏輯閘元件湊出(組合出)所需要的電路。

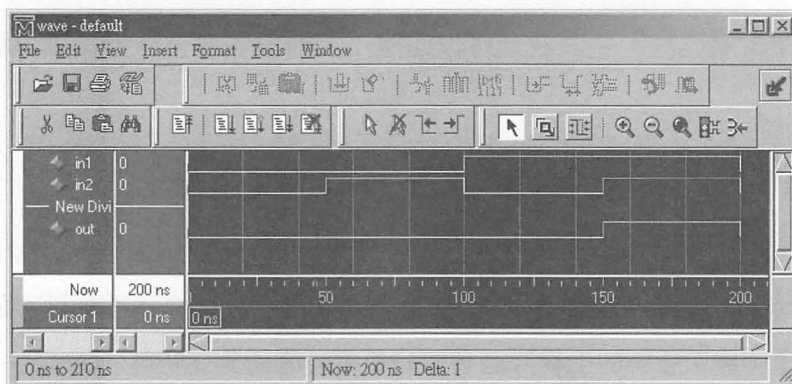
例：and, nand, or, nor, xor, xnor, buf, not, bufif1, bufif0, notif1, notif0 ... 等邏輯閘元件。



▲ 2 輸入的 AND 閘

```
// GATEAND2.V, 使用邏輯閘層次來描述 1 個 2 輸入的 AND 閘
module GATEAND2 (in1, in2, Out);
input  in1, in2;
output Out;
wire  Out;

    and  u1 (Out, in1, in2);
endmodule
```



▲ GATEAND2.V 的模擬結果

(三) 資料處理模型 (Data Flow Model) 或 暫存器轉移層次 (Register Transfer Level)

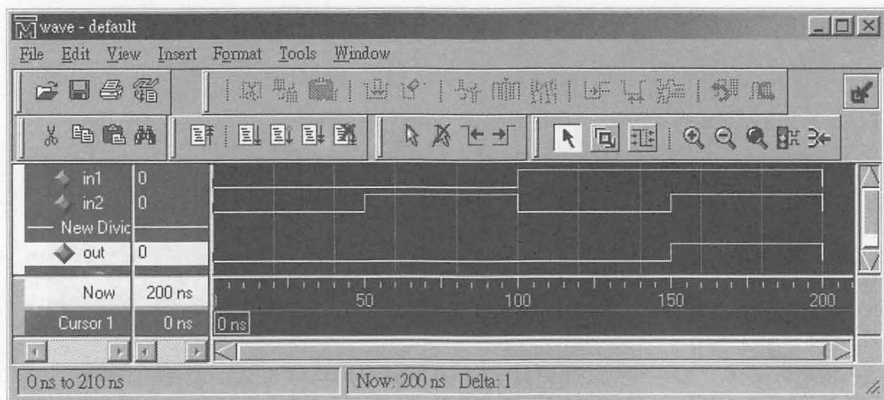
在這個層次中，設計電路的重點在於說明資料如何在暫存器中儲存與傳送。

例：使用 assign 述產生的指定動作。

```
// DF_AND2.V, 使用資料處理模型層次來描述 1 個 2 輸入的 AND 閘
module DF_AND2 (in1, in2, Out);
input  in1, in2;
```

```
output Out;
wire Out;

assign Out = in1 & in2;
endmodule
```



▲ DF_AND2.V 的模擬結果

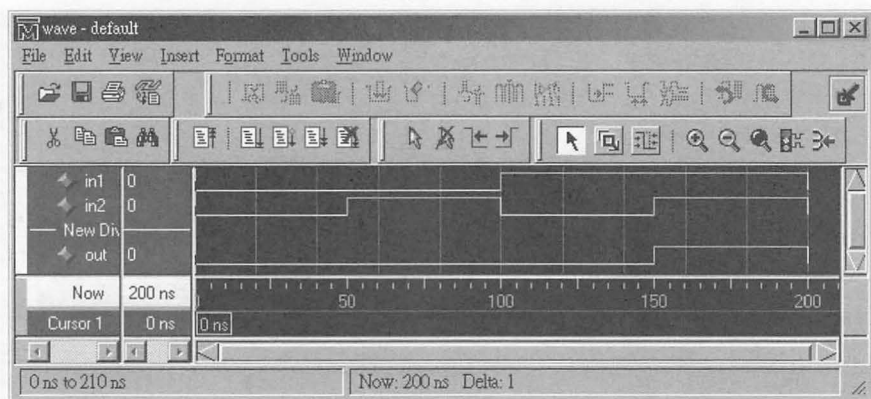
(四) 行為模型 (Behavioral Model)

這個層次是 Verilog HDL 中最高階的層次，在這個層次中我們只需要考慮模組的功能，並不需要考慮元件的物理特性、連接線路的特性...等等關於硬體方面詳細的部份。相較於電晶體層次，在這個層次電路設計的工作就好像是在寫 C 語言一樣，是一種比較高階的設計方式。

例：使用 `always`, `for`, `while`, `case`, `casex`, `casez` ... 等敘述產生的電路。

```
// BEH_AND2.V, 使用行為模型層次來描述 1 個 2 輸入的 AND 閘
module BEH_AND2(in1, in2, Out);
input in1, in2;
wire in1, in2;
output Out;
reg Out;
```

```
always @(in1 or in2)
begin
    Out = in1 & in2;
end
endmodule
```



▲ BEH_AND2.V 的模擬結果



3.5 Verilog 的模組 (Module)

模組 (Module) 是組成一個電路的最小單位，它描述了該：模組的名稱、所需的輸出入埠的名稱、輸出入埠的個數、輸出入埠的大小、電路中所需的接線及暫存器、引用較低階之模組的別名、電路所需功能的指定描述 (assign)、電路所需功能的行為層級之描述、函數 (function)，以及任務 (task) 等。

在 Verilog 中模組的宣告是用關鍵字 `module` 和關鍵字 `endmodule`。一個模組的開頭永遠是 `module` 這個關鍵字，接下來在 `module` 後需加一個用以識別的模組名稱 (module name)，接著宣告模組的輸入與輸出的埠列 (module terminal list)，以及視情況而決定是否需要參數的宣告。在模組的最後要加上 `endmodule` 代表模組的結尾。

只有在當模組需要與外界相互連接時才有埠列與埠的宣告。其他的五個部份包括：`wire` 與 `reg` 等變數的宣告、引用較低階之模組的別名、`assign` 資料處理層級之描述、`always` 行為層級之描述、以及 `task` 任務與 `function` 函數，這五大部份並沒有有一定的撰寫順序，一個模組的尾端永遠是 `endmodule` 這個關鍵字。

```
module 模組名稱
埠列與埠的宣告 (input, output, inout ...等)

wire, wand, wor 與 reg ... 等資料型態變數的宣告

引用較低階之模組的別名

assign 資料處理層級之描述

always 行為層級之描述區塊
begin
// 資料處理與指定 ... 等描述 或
// task 任務 與 function 函數 的使用
end

function 函數 或 (以及) task 任務的宣告

endmodule
```

在一個 Verilog 檔案 (*.V 檔) 中可以擁有許多個模組，且每個模組在檔案中並沒有像 C 語言一樣規定：會被引用到的函數要寫在引用之前的位置或在前頭宣告的地方，Verilog 模組宣告的順序可以是任意的、不受限於引用層次的高低。

模組是 Verilog 的基本組成元件，一個模組可以是一個基本元件或是由其他的模組所組合而成的。我們只需藉由連接模組與模組間的介面、輸出埠與輸入埠及雙向埠，就可以設計所需要的元件，而不需要去考慮每個模組內部的詳細電路是什麼。這使得設計模組內部電路的人可以放心地修改其內部的電路，而不必擔心對於設計過程中其他部份所造成的影響。

(一) 模組的 BNF 表示法

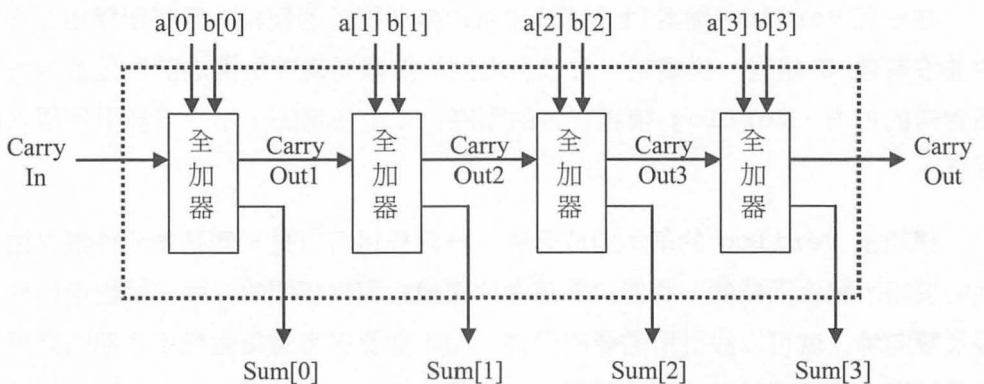
```
<module> :=  
module <module_name>  
<module_ports>? ;  
<module_items>*  
endmodule
```

(二) 模組的「別名」

模組就如同一個模子一樣，我們能用這個模子來創造出所需要的物件。每個從模子中創造出來的物件都是獨立的，有其自己的名字、變數、參數與輸出入埠介面的宣告。這個從模組創造物件的過程我們稱為：為模組取「別名」。

底下的範例是一個 4 個位元的漣波進位加法器 (4 Bits Ripple Carry Adder)，最上層的區塊擁有 4 個從 FullAdd 模組中創造出來的「別名」，分別為：fa0、fa1、fa2 及 fa3；每個別名均包含了構成 FullAdd 模組所需要的基本邏輯敘述。

範例 4 位元的漣波進位加法器 (4 Bits Ripple Carry Adder)



▲ 4 位元的漣波進位加法器 (4 Bits Ripple Carry Adder) 的邏輯符號

【4 位元的漣波進位加法器 (4 Bits Ripple Carry Adder) 的電路描述】

```
// FullAdd4.V, 4 位元漣波進位計數器
module FullAdd4 (a, b, Carry_In, Sum, Carry_Out);
input      [3:0] a, b;
input      Carry_In;
output     [3:0] Sum;
output     Carry_Out;
.wire      [3:0] Sum;
wire       Carry_Out;
  FullAdd fa0 (a[0], b[0], Carry_In, Sum[0], Carry_Out1);
  FullAdd fa1 (a[1], b[1], Carry_Out1, Sum[1], Carry_Out2);
  FullAdd fa2 (a[2], b[2], Carry_Out2, Sum[2], Carry_Out3);
  FullAdd fa3 (a[3], b[3], Carry_Out3, Sum[3], Carry_Out );
endmodule
```

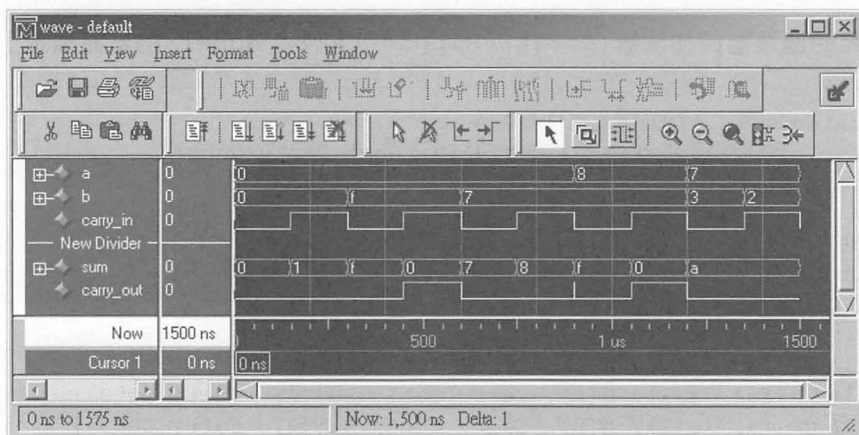


▲ 1 位元全加器 (1 Bits Full Adder) 的邏輯符號

【1 位元全加器 (1 Bits Full Adder) 的電路描述】

```
module FullAdd (a, b, CarryIn, Sum, CarryOut);
input  a, b, CarryIn;
output Sum, CarryOut;
wire   Sum, CarryOut;

  assign {CarryOut, Sum} = a + b + CarryIn;
endmodule
```



▲ 4 位元的漣波進位加法器（4 Bits Ripple Carry Adder）的模擬結果



3.6 Verilog 的語法協定

Verilog 是由一連串的標記(token)所組成，這些標記可以是空白(White-space)、註解(Comments)、運算子(Operators)、數值(Numbers)、識別字(Identifiers)與關鍵字(Keywords)及底線(Underscore characters)和問號(Question Marks)。詳細的說明，如下：

- (一) 識別字(Identifiers)的大小寫是不同的，例：Even 和 even 是二個不同的標記名稱。
- (二) 所有的關鍵字(Keywords)名稱必須使用小寫來表示。
- (三) 空白 (Whitespace)

Verilog 對空白的定義為：只要是空格(Blank Space, ' ')、跳格(Tab, '\t')和換行符號(New Line, '\n')皆屬於空白字元。

(四) 註解 (Comments)

可以 Verilog 電路描述檔中特定的加入對此模組的說明，便於日後再閱讀或稱修改此模組時，可以很快地了解這個模組的功能及模組中特定描述的說明資料，這個說明的部份就稱之為註解。註解的寫法，有下列二種：

- 以 “//” 為開頭的註解寫法，稱為「單行註解」(One Line Comment)。

例：

```
// FullAdd4.V, 4 位元漣波進位計數器
wire a = b & c; // 宣告 1 條接線 a, 並指定它為 b & c
```

- 以 “/*” 為開頭，並以 “*/” 當作結尾的註解寫法，稱為「多行註解」(Multiple Line Comment)。

要注意的是：多行註解內不可以再有多行註解。

例：

```
/* 若 op = ADD, 則 Result = a + b; 若 op = SUB, 則 Result = a + b;
   若 op = AND, 則 Result = a & b; 若 op = OR, 則 Result = a | b;
   若 op 不為上述 4 種之 1 的話, 則 Result = a; */
/* 錯誤的多行註寫法
   /* 因為多行註解內有另一個多行註解 */ */
```

(五) 運算子 (Operators)

運算子有下列三種格式：

- 單元 (Unary) 運算子

單元運算子必須放在運算元的前面。

例：

```
assign a = ~b; // ~, Bit-wise NOT
```

■ 二元 (Binary) 運算子

二元運算子放在二個運算元之間。

例：

```
assign a = b & c; // &, Bit-wise AND
```

■ 三元 (Ternary) 運算子

三元運算子就是條件運算子 (Conditional operators)、(? :)。

BNF 表示法

```
Value = Test _Condition ? True_Part : False_Part;
```

說明

如果 Test Condition 的值為 1 (True, 真) 的話，則 Value 的值將被設定為 True_Part 的值，否則 Value 的值將被設定為 False_Part 的值。

例：

```
assign Out = Select ? a : b;
// 如果 Select = 1, 則 Out = a, 否則 Out = b
```

(六) 數值 (Numbers)

數值規格分為：定長度 (Sized) 數字，以及不定長度 (UnSized) 數字。

■ 定長度 (Sized) 數字

定長度數字是以 `<size>'<base format><number>` 來表示。其中 `<size>` 是以十進位來表示數字的位元數 (Bits)，`<Base Format>` 用來定義此數值為：十六進位 ('h 或 'H)、十進位 ('d 或 'D)、八進位 ('o 或 'O)，或者是二進位 ('b 或 'B)。

其中：十六進位 ('h 或 'H) 可用的數字為 0~9 及 A~F (A、B、C、D、E 及 F)，其中 A~F 也可用小寫的 a~f (a、b、c、d、e 及 f) 來表示。

例：

```

input    [2:0] in;
wire     [31:0] Out1, [13:0] Out2, [12:0] Out3, [7:0] Out4;

// 32 Bits 的十六進位數值，左移 in 次
assign   Out1 = 32'h89AbCDef << in;
等於     assign Out1 = 32'h89Ab_CDef << in;
其中 _ 底線符號在此只是方便您對此數值的讀取

// 14 Bits 的十進位數值，左移 in 次
assign   Out2 = 14'd1289 << in;
等於     assign Out2 = 14'd1_289 << in;

// 13 Bits 的八進位數值，左移 in 次
assign   Out3 = 13'o3777 << in;
等於     assign Out3 = 13'o3_777 << in;

// 8 Bits 的二進位數值，左移 in 次
assign   Out4 = 8'b10101010 << in;
等於     assign Out4 = 8'b1010_1010 << in;

```

■ 不定長度 (UnSized) 數字

不定長度數字不必使用 `<size>` 來規定數字之位元數大小，而是使用 HDL 編譯器內定的長度。其中若沒有寫明 `<base format>` 的部份，則此數值內定為十進制。

例：

```

assign Out1 = 'h89Ab << in;    // 十六進位數值，左移 in 次
assign Out2 = 128 << in;        // 十進位數值，左移 in 次
assign Out3 = 'o377 << in;      // 八進位數值，左移 in 次
assign Out4 = 'b10101010 << in; // 二進位數值，左移 in 次

```

(七) 識別字 (Identifiers) 與關鍵字 (Keywords)

識別字是在 Verilog 電路描述中所給予物件的名稱。識別字的第一個字元必須為字母，第二個之後的字元可為字母、數字、底線 “_” 或是錢號 “\$” 所組成。識別字大小寫是有分別的，例：`Even` 和 `even` 是二個不同的標記名稱。

關鍵字是一組特殊的定義名稱，其功用是為了定義 Verilog 電路描述的架構。所有的關鍵字 (Keywords) 名稱必須使用小寫來表示。

例：

```
input    CarryIn; // input 是關鍵字, CarryIn 是識別字
wire     CarryOut; // wire 是關鍵字, CarryOut 是識別字
```

Verilog 的關鍵字 (Keywords) 有：module、endmodule、begin、end、always、assign、input、output、inout、wire、reg、parameter、buf、bufif0、bufif1、not、notif0、notif1、and、nand、or、nor、xor、xnor…等等都是用英文小寫來表示的。

(八) 底線 (Underscore characters)、x 以及 ? 問號 (Question Marks)

底線 “_” 的目的在於增加模組名稱、變數名稱、function 的名稱以及 task 的名稱或關鍵字…等等的可讀性。必須注意的是上述這些名稱或關鍵字的第一個字元不能是底線。

底線符號 “_” 也同樣具有增加數值的可讀性之作用。

x 表示 Unknown (未知的值)，而 ? 表示 High Impedence (高阻抗)。

例：

```
wire    [11:0] Out1,  [7:0] Out2;
wire    [3:0] Out_X,  [1:0] Out_Z;

// 12'b1010_1111_0101 等於 12'b101011110101
assign Out1 = 12'b1010_1111_0101 << in;

// 8'b11??11?? 等於 8'b11zz11zz
assign Out2 = 8'b11??11?? << in;
```

```
// 4 Bits 的二進位數值且給定 Unknown
assign Out_X = 4'bx;

// 2 Bits 的二進位數值且給定 High Impedence
assign Out_Z = 2'b?;
```



3.7 階層式設計 (Hierarchy Design) 的觀念

階層式設計 (Hierarchy Design) 觀念所要描述的結構和資料結構中的樹狀表示大同小異，這種設計方式又可分為：由低至高的設計方式 (Bottom-up design)，以及由高至低的設計方式 (Top-Down design)。基本上這二種設計方式最終都會需要幾個較基本的模組，因為任何一個較具規模的設計都可由幾個較基本的模組所組成。

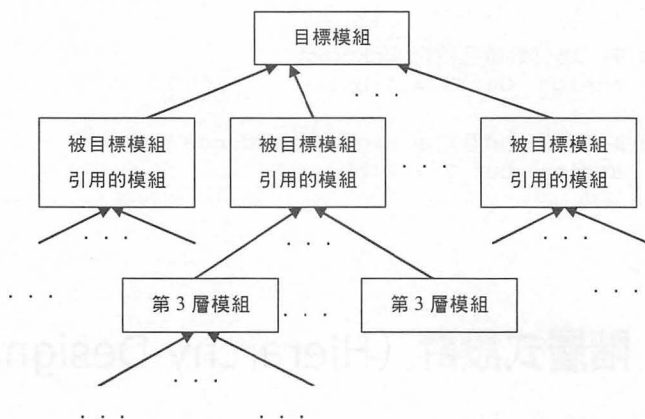
3.7.1 由低至高與由高至低的設計方式

(一) 由低至高的設計方式 (Bottom-up design)

這種設計模組的方式是先將一些常用的模組設計出來之後，再用這些模組來設計出最後的目標模組。因為一個較具規模的設計是由幾個較基本的模組所組成的，像是堆積木的觀念：由小到大地設計出想要的目標模組。

目標模組是由一個或以上被目標模組引用之模組所組成的，而其中的某一個被目標模組引用的模組又是由一個或以上的第 3 層模組所組成的。

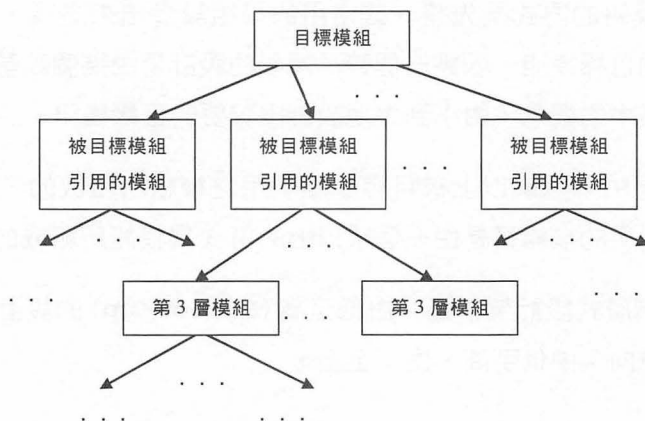
如下圖「階層式設計架構圖－由低至高 (Bottom-up) 的設計方式」所示，請注意箭頭的方向：由低至高、由下至上。



▲ 階層式設計架構圖—由低至高（Bottom-up）的設計方式

（二）由高至低的設計方式（Top-Down design）

這種設計模組的方式是先開出目標模組的功能要求之後，去畫分出幾個較大功能的區塊模組，再去分析組成各個較大功能的區塊模組所需要用到的第 3 層及較低層次的模組，最後再去設計第 3 層及較低層次的這些模組。有了第 3 層及較低層次的這些模組就可以結合出第 2 層次的模組，所需的第 2 層次的模組都合成出來之後，就可以結合出目標模組了。這種設計模組的觀念，傾向於功能導向式模組設計觀念。

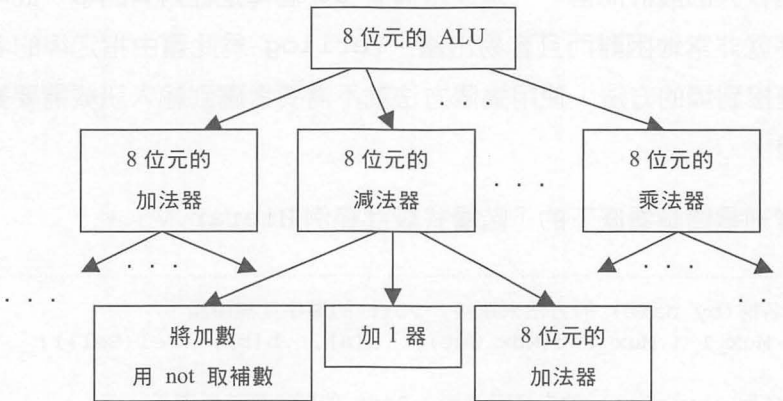


▲ 階層式設計架構圖—由高至低（Top-Down）的設計方式

如上圖「階層式設計架構圖－由高至低 (Top-Down) 的設計方式」所示，請注意箭頭的方向：由高至低、由上至下。

範例 想要設計一個 8 位元的 ALU，由高至低 (Top-Down) 的設計方式

一個 8 位元的 ALU，可由一個 8 位元的加法器、8 位元的減法器、8 位元的乘法器...等等模組所組成，而其中 8 位元的減法器又可由一個具有：用 not 閘將加數取補數後加 1 的模組，以及一個 8 位元的加法器等模組所組成。



3.7.2 埠對應 (Port Mapping) 連接的方式

將一個模組的埠與外部訊號連接的方法有兩種，分別為：依照定義模組時埠列的「順序」(in order)來連接，以及依「指定名稱」(by name)的方法來連接。這兩種方法必須分開使用，不可以同時在同一個引用模組的別名中使用。

(一) 依照定義模組時埠列的「順序」(in order) 來連接

這個方法對於初學 Verilog 的人來說是最直覺的方法，將外部訊號依照定義這個模組時的埠列順序接到引用這個模組的別名。

例：詳細電路請看底下的「階層式設計範例 Hierar.V」。

```
Mux      Mux_1 (Sel, a, b, Mux_Out);
Register8 Register8_1 (Clock, Reset, Mux_Out, Reg_Out);
Rotate_Data Rotate_Data_1 (Clock, Reset, Left_Rotate,
Reg_Out, out);
```

(二) 依照「指定名稱」(by name) 的方法來連接

對一個較大的設計而言，一個模組擁有 50 個埠是經常有的事，此時要詳記埠列的順序就非常地困難而且容易出錯。Verilog 為此藉由指定埠的名稱，將外界訊號連接到埠的方法，使用這個方法就不需要考慮到輸入訊號需要對應埠列順序的問題。

例：詳細電路請看底下的「階層式設計範例 Hierar.V」。

```
// 依指定名稱(by name) 的方法來連接, Port 的順序就無所謂了
Mux      Mux_1 (.Mux_Out(Mux_Out), .a(a), .b(b), .Sel(Sel));

// 依指定名稱 (by name) 的方法來連接, Port 的順序就無所謂了
Register8 Register8_1 (.Clock(Clock), .Reset(Reset),
                      .Mux_Out(Mux_Out), .Reg_Out(Reg_Out));

// 依照定義模組時埠列的順序(in order) 來連接
Rotate_Data Rotate_Data_1 (Clock, Reset, Left_Rotate,
Reg_Out, out);
```

【Hierar.V：階層式設計範例】

```
// Hierar.V：階層式設計範例
// 首先定義較小的模組：Mux, Register8 及 Rotate_Data 之後再去引用它們
// Multiplexor 多工器
module Mux (Sel, a, b, out);
input      Sel;
input      [7:0] a, b;
output     [7:0] out;
```

```
wire      [7:0] out;

    assign out = Sel ? a : b;
endmodule

// 8 個位元的暫存器
module Register8 (Clock, Reset, data, q);
input      Clock, Reset;
input      [7:0] data;
output     [7:0] q;
reg        [7:0] q;

    always @(posedge Clock)
    begin
        if (Reset)
            q = 0;
        else
            q = data;
        end
    end
endmodule

// 「載入」資料或「旋轉」資料
module Rotate_Data (Clock, Reset, Left_Rotate, data, q);
input      Clock, Reset, Left_Rotate;
input      [7:0] data;
output     [7:0] q;
reg        [7:0] q;

    always @(posedge Clock)
    begin
        if (Reset) // 重置
            q = 8'b0; // 將 q 清除為 0
        else
            if (Left_Rotate) // 「旋轉」資料
                q = {q[6:0], q[7]};
            else
                q = data; // 從輸入埠 data, 「載入」資料至 q
            end
        end
    end
endmodule
```

【依照定義模組時埠列的「順序」(in order)來連接】

```
// -----
// 模組 Top1：依照定義模組時埠列的「順序」(in order) 來連接
// -----
module Top1 (Clock, Reset, Sel, Left_Rotate, a, b, out);
input      Clock, Reset, Sel, Left_Rotate;
input      [7:0] a, b;
output     [7:0] out;
wire       [7:0] Mux_Out, Reg_Out;

    Mux      Mux_1 (Sel, a, b, Mux_Out);
    Register8 Register8_1 (Clock, Reset, Mux_Out, Reg_Out);
    Rotate_Data Rotate_Data_1 (Clock, Reset, Left_Rotate, Reg_Out,
out);

endmodule
```

【依照「指定名稱」(by name)的方法來連接的電路描述部份】

```
// -----
// 模組 Top2：依照「指定名稱」(by name) 的方法來連接
// -----
module Top2 (Clock, Reset, Sel, Left_Rotate, a, b, out);
input      Clock, Reset, Sel, Left_Rotate;
input      [7:0] a, b;
output     [7:0] out;
wire       [7:0] Mux_Out, Reg_Out;

    // 依 「指定名稱」(by name) 的方法來連接, Port 的順序就無所謂了
    Mux      Mux_1 (.Sel(Sel), .a(a), .b(b), .out(Mux_Out));

    // 依 「指定名稱」(by name) 的方法來連接, Port 的順序就無所謂了
    Register8 Register8_1 (.Clock(Clock), .Reset(Reset),
        .data(Mux_Out), .q(Reg_Out));

    // 依照定義模組時埠列的「順序」(in order) 來連接
    Rotate_Data Rotate_Data_1 (Clock, Reset, Left_Rotate, Reg_Out, out);

endmodule
```

本章練習

Question 1 :

什麼時候用 `assign` 敘述？什麼時候用 `always` 敘述？

Question 2 :

如果您的電路需要 $32K \times 16$ 的記憶體空間用來儲存資料、用陣列 (Arrays) 表示法來描述與用 FPGA 或 CPLD 提供的記憶體區塊，試著說明二者在電路面積與存取速度上的差異。

Question 3 :

邏輯閘層次模型 (Gate Level Model)、資料處理模型 (Data Flow Model) 或暫存器轉移層次 (Register Transfer Level) 與行為模型 (Behavioral Model) 的使用時機各是？請舉例說明。

Question 4 :

由低至高的設計方式 (Bottom-up design) 與由高至低的設計方式 (Top-Down design) 的使用時機分別是？請舉例說明。

Question 5 :

通常會採用那一種埠對應 (Port Mapping) 連接的方式？那一種比較不會出錯？請舉例說明。

PS Google 可以幫您找到解答。

